
SmallFloats.jl Compact Guide

0.4.0

Jeffrey Sarnoff

2026-08-21T19:54Z

Contents

Contents		ii
1	SmallFloats.jl	1
1.1	Installation	1
1.2	A first check	1
1.3	What the package guarantees	2
1.4	Contents	2
2	Formats and Values	3
2.1	The format space	3
2.2	A format, listed in full	4
2.3	Construction projects	4
2.4	The code-point rule	5
2.5	Predicates, stepping, classification	5
2.6	Display	5
2.7	Next	5
3	Computing	6
3.1	One projection, named in the call	6
3.2	Rounding and saturation	6
3.3	Stochastic rounding	7
3.4	Random values	7
3.5	Session defaults	8
3.6	Interaction with Julia's numeric model	8
3.7	Next	8
4	Data at Scale	9
4.1	Arrays and tables	9
4.2	Choosing a format	9
4.3	Blocks	10
4.4	Packed storage	11
4.5	Float16 and BFloat16	11
4.6	Next	12
5	A Worked Example	13
5.1	The data, and an explicit policy	13
5.2	Measure, with three numbers	13
5.3	Compare candidates under one policy	14
5.4	Inspect the worst misses	14
5.5	Record what produced the result	14
5.6	Next	14
6	Assurance	15
6.1	The provenance test	15
6.2	Measured approximation	15
6.3	The conformance declaration	16
6.4	Verifying your own code	16
6.5	How the defined path is verified	17
6.6	Next	17

7	Quick Reference	18
7.1	Values in and out	18
7.2	Format queries	18
7.3	Projection specifications	19
7.4	Operations	19
7.5	Session defaults	19
7.6	Display	19
7.7	Blocks, storage, conformance	19
7.8	Common mistakes	20
8	Troubleshooting	21
9	Glossary	22

Chapter 1

SmallFloats.jl

A compact guide to the package. The [full manual](#) covers workflows, complete reference tables, and implementation detail.

SmallFloats.jl implements the IEEE P3109 draft standard, *Arithmetic Formats for Machine Learning*. It provides all 504 legal formats at bitwidths 3 through 16, with bit-exact defined results and explicit control over rounding policy.

In formats this small, most operations round: the numbers you construct and the results you compute are usually not representable. The package makes the rounding visible and controllable rather than implicit. Three definitions carry the whole model:

- A **format** is a finite set of exact values, called *datums*, each named by an integer *code point*.
- A **value** stores one code point. Reading the datum back with `decode` is always exact.
- An **operation** decodes its operands, computes the exact mathematical result, and applies one *projection* — a rounding choice plus a saturation choice — to produce the result code point.

Each chapter of this guide develops part of that model.

1.1 Installation

Requires Julia 1.12 or later. The package is not yet registered; install by URL:

```
pkg> add https://github.com/JeffreySarnoff/SmallFloats.jl
```

Quadmath (Float128) is a hard dependency. It is used only inside exact oracle and fallback paths, never anywhere it could change a result.

1.2 A first check

```
julia> using SmallFloats

julia> Binary8p4se(1.6) + Binary8p4se(0.25)
Binary8p4se(1.875 ≈ 0x47)

julia> conformance() !== nothing
true
```

The first line exercises the projection engine: 1.6 is not representable, so construction rounds it to 1.625, and the sum lands exactly on 1.875. The second line confirms the conformance machinery is available. Values in this guide display as `value ≈ code point`, showing both halves of every result.

1.3 What the package guarantees

Four properties distinguish the package. They are stated plainly here and developed in the chapters that follow.

No hidden rounding. An operation computes its exact mathematical result and rounds it exactly once. There is no intermediate precision to accumulate error inside an operation, and no fast path that substitutes a different answer: the code point you receive is the one the standard defines, every time.

Reproducible policy. The rounding rule is not something you must remember about the session — it can be written into each call. Code that names its rule produces identical results in every session, on every machine.

No silent approximation. Faster, inexact implementations exist only if you register one, by name. Registration measures the implementation's worst-case error over every input and rejects any declared bound that understates the measurement. Nothing approximate can be reached by accident.

One behavior at every size. All 504 formats, from 3 bits to 16, follow the same rules. A wider format costs more storage; it never means something different.

1.4 Contents

[Formats and Values](#): the format space, construction, and the code-point rule. [Computing](#): the projection contract and the policy choices. [Data at Scale](#): choosing a format, arrays, blocks, packed storage, and IEEE interop. [A Worked Example](#): the chapters composed into one measured quantization task. [Assurance](#): conformance export, measured approximation, and verification. Three lookups close the guide: [Quick Reference](#), [Troubleshooting](#) (symptom-indexed), and a [Glossary](#).

Chapter 2

Formats and Values

This chapter defines the format space, shows how a value names a datum, and states the constructor rule that prevents the most common usage error.

2.1 The format space

Every format is an instance of one parametric type, `Binary{K,P,SGN,EXT}`: bitwidth $K \in 3:16$, precision P (significant bits, implicit bit included), SGN for signed or unsigned, EXT for extended (with $\pm\text{Inf}$) or finite. Each legal combination — 504 in total — has a concrete alias whose name encodes the parameters: `Binary8p4se` is 8-bit, precision 4, signed, extended.

The bitwidth is a fixed budget, spent entirely:

One sign bit if signed, $E = K - P + 1 - SGN$ exponent bits, and $P - 1$ stored significant bits. Raising precision by one bit removes one exponent bit; choosing a format is choosing this trade.

using `SmallFloats` exports the 120 aliases at $K \leq 8$. The other 384 are available three ways — fully qualified, via `using SmallFloats.Formats`, or programmatically:

```
julia> T = format(16, 6, true, true)
Binary16p6se

julia> codeunit_type(T), SmallFloats.datumcarrier(T)
(UInt16, Float64)
```

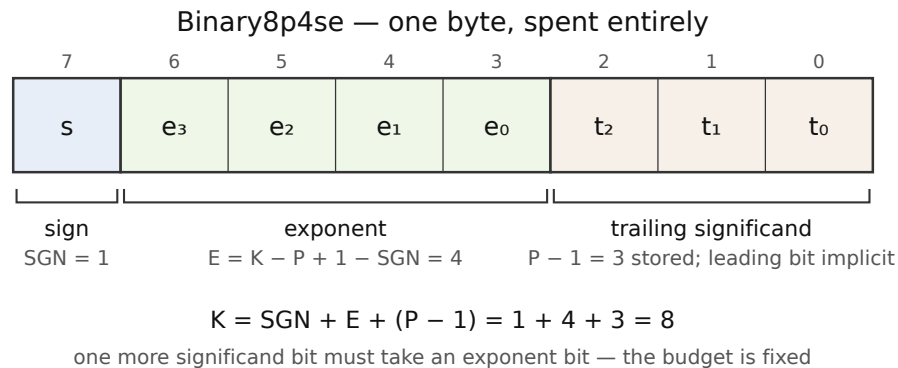


Figure 2.1: Bit-field anatomy of `Binary8p4se`: one sign bit, four exponent bits, three stored significand bits, with the leading significand bit implicit.

Code points occupy a `UInt8` at $K \leq 8$ and a `UInt16` above. `decode` returns the format's *datum carrier*: `Float64` for the 432 formats it holds exactly, `Float128` or an exact dyadic carrier for the other 72. Both are representation details; the value model does not change with K .

One practical caution. The abstract type `Binary{8,4,true,true}` is a supertype of the concrete `Binary8p4se`, and it is not a valid array element type: `Vector` of the abstract format boxes every element and loses all of the performance machinery. Use the concrete alias, or `format(...)`, in element types.

2.2 A format, listed in full

Small formats can be inspected completely. `Binary4p2se` has 16 code points, and this listing is the entire format:

```
julia> decode.(sort(Binary4p2se.(0x00:0x0f)))
16-element Vector{Float64}:
 NaN
 -Inf
 -2.0
 -1.5
 -1.0
 -0.75
 -0.5
 -0.25
  0.0
  0.25
  0.5
  0.75
  1.0
  1.5
  2.0
  Inf
```

The same table for any format, in storage order or total order, is a few lines. No projection is involved — `decode` is exact and policy-free, so the table depends on the format alone:

```
function codetable(::Type{F}; by::Symbol = :code) where {F<:Binary}
    U = codeunit_type(F) # UInt8 at K ≤ 8, UInt16 above
    vals = [F(U(c)) for c in 0:(1 << bitwidth(F)) - 1]
    by == :value && sort!(vals) # total order: NaN first
    [(code = codepoint(v), value = decode(v)) for v in vals]
end
```

`codetable(Binary4p2se)` lists code `0x08` — the sign bit alone — as the single `NaN`, with negatives mirroring the positives above it; `codetable(Binary4p2se; by = :value)` reproduces the sorted listing above. The reverse table, `value → code`, does need a projection: `codepoint(Convert(F, p, x))`, one rounding per entry.

Three P3109 design decisions are visible here. There is exactly one `NaN`, with no payloads and no quiet/signalling distinction. There is no negative zero; `T(-0.0)` projects to the single zero. And `NaN` sorts first, below `-Inf`: the draft's total order places it at the opposite end from `Float64` sorting, which gives sorting and searching deterministic behavior. Finite formats (suffix `f`) assign the two infinity code points to two additional finite datums.

2.3 Construction projects

```
julia> x = Binary8p4se(1.6)
Binary8p4se(1.625 ≈ 0x45)
```

The number 1.6 is not among the format's values. Its nearest neighbors are 1.5, 1.625, and 1.75, spaced 0.125 apart in this binade, so construction rounds to the nearest datum and stores that datum's code point. Constructors from a `Real` never throw for representable magnitudes; they round.

decode and codepoint read back the two halves:

```
julia> decode(x), codepoint(x)
(1.625, 0x45)
```

decode never rounds; it is a lookup from code point to datum. The asymmetry matters: reading is exact, writing projects.

2.4 The code-point rule

An Unsigned argument means code point. Every other Real argument means a value to project. The integer type's signedness is the entire distinction:

```
julia> Binary8p4se(0x02), Binary8p4se(2)
(Binary8p4se(0.001953125 ≈ 0x02), Binary8p4se(2.0 ≈ 0x48))
```

The same digit, read two ways, differs by a factor of 1024 here — and neither call is an error, so a wrongly-typed integer produces a valid, different result rather than a failure. Use unsigned integers when you mean stored identity (a byte read from a file, a wire format). Use signed integers or any other Real when you mean a number. When intent must be explicit regardless of the argument's type, `Convert(T, p, x)` always projects numerically.

The round-trip law holds in one direction: `T(codepoint(x)) == x` for every valid code point. The other direction fails by design — `decode(T(1.6))` returns 1.625, because construction already chose a datum, and nothing downstream can recover the original argument.

2.5 Predicates, stepping, classification

The standard numeric vocabulary applies to every format: `isnan`, `isinf`, `isfinite`, `iszero`, `signbit`, `issubnormal`, along with `zero`, `one`, `eps`, `typemin`, and `typemax`, all answering from the format's own grid. Stepping moves to the adjacent code point, and `Class` reports the draft's classification:

```
julia> NextGreaterThan(Binary8p4se(1.0))
Binary8p4se(1.125 ≈ 0x41)

julia> Class(Binary8p4se(0.01))
ClassPosNormal::FPClass = 0x06
```

`NextGreaterThan` and `NextLessThan` are the draft's Next operations; `nextfloat` and `prevfloat` alias them.

One related distinction: Base's lowercase `convert` is value-preserving on Unsigned input — `convert(Binary8p4se, 0x02)` is the value 2.0, not code point 2 — because Julia's promotion machinery, `similar`, and `fill` depend on that behavior. The code-point reading belongs to the constructor alone. Prefer `T(x)` or `Convert(T, p, x)` in your own code and leave bare `convert` to the places Base itself calls it.

2.6 Display

Four styles render the same value: `:value` (1.625), `:codepoint` (0x45), `:datum` (1.625 ≈ 0x45), and `:typed` (Binary8p4se(1.625 ≈ 0x45)). Select one process-wide with `set_show_style!` or per-stream with the `:binary_show_style` I/O context property. Display is presentation only; no kernel or comparison reads it. This guide uses `:typed` throughout.

2.7 Next

Values exist but nothing has computed with them. Construction from a Real rounded, and this chapter did not say by what rule. The rule is a policy the caller selects, and it is the subject of [Computing](#).

Chapter 3

Computing

Every numeric operation follows the same three steps: decode the operands, compute the mathematically exact result over the extended reals, and project once. There is no arithmetic "in the format" and no accumulated rounding inside an operation. Rounding error arises only at the single point where the exact result becomes a code point. This chapter covers that point: the two policy choices that decide it, and how to keep the decision stable.

3.1 One projection, named in the call

```
julia> x, y = Binary8p4se(1.6), Binary8p4se(0x46);  
  
julia> Add(Binary8p4se, RNE_SN, x, y)  
Binary8p4se(3.5 ≈ 0x4e)
```

The call reads left to right: operation, result format, projection specification, operands. The exact sum, $1.625 + 1.75 = 3.375$, is not a datum; it lies exactly halfway between 3.25 and 3.5, so the stored result is determined entirely by policy. `RNE_SN` rounds to nearest with ties to even, selecting 3.5, then applies the draft's `SatNone` boundary rules. A different projection could land the same sum on 3.25. The projection therefore appears in the argument list: it is part of the operation, not an ambient setting.

The ordinary Julia spelling uses the current session default:

```
julia> x + y  
Binary8p4se(3.5 ≈ 0x4e)
```

Use `+` and the other operators for interactive work. Use `Add(T, p, ...)` in code whose results must not depend on session state.

3.2 Rounding and saturation

A `ProjSpec` pairs two independent choices. Rounding selects the neighbor: six deterministic modes (nearest-ties-even, nearest-ties-away, toward positive, toward negative, toward zero, to-odd), with every pairing predefined under names such as `RNE_SN` and `RTZ_SF`. Saturation decides what an out-of-range result becomes: `SatFinite` clamps to the finite extremes, `SatPropagate` preserves representable infinities, and `SatNone` applies the draft's full domain-, sign-, and direction-dependent rows. `SatNone` does not mean "no saturation"; it means neither simplifying policy was selected.

The axes separate cleanly on one overflow. Note first that `Binary8p4se(200.0)` already projected during construction — 200 ties between 192 and 208 and lands on 192 — so both multiplies below compute the exact product 384, which exceeds the largest finite datum, 224:

```
julia> w, two = Binary8p4se(200.0), Binary8p4se(2.0);

julia> Multiply(Binary8p4se, RNE_SN, w, two)  # nearest + draft rows → Inf
Binary8p4se(Inf ≈ 0x7f)

julia> Multiply(Binary8p4se, RNE_SF, w, two)  # nearest + clamp → 224
Binary8p4se(224.0 ≈ 0x7e)

julia> Multiply(Binary8p4se, RTZ_SN, w, two)  # toward zero + draft rows → 224
Binary8p4se(224.0 ≈ 0x7e)
```

The first two calls differ in saturation; the first and third differ in rounding. The third is the subtle case: rounding runs first and knows nothing about range, and saturation then classifies the out-of-range result with the rounding mode passed in — so a directed mode pointing away from the overflow yields the finite extreme even under `SatNone`. Choose `SatNone` when an overflow should remain observable as an infinity, `SatFinite` when downstream code cannot absorb one.

3.3 Stochastic rounding

The three stochastic families (`StochasticA/B/C`, with random-bit budget N) make the landing depend on the exact fractional position and a draw $R \in 0:2^N-1$. The distribution is part of the definition: projecting $2 + 3/64$ under `RSA_SN(4)`, a value three sixteenths of the way from 2.0 to 2.25, rounds upward for exactly 3 of the 16 possible draws. Fixing R makes a single projection exactly reproducible, which suits tests:

```
julia> σ = RSA_SN();  # StochasticA, default N = 8

julia> a, b = Binary8p4se(2.0), Binary8p4se(0.03125);

julia> Add(Binary8p4se, σ, a, b; R = 0)
Binary8p4se(2.0 ≈ 0x48)  # smallest draw: rounds down

julia> Add(Binary8p4se, σ, a, b; R = 255)
Binary8p4se(2.25 ≈ 0x49)  # largest draw: rounds up
```

For streams, pass a seeded rng; stochastic projection draws from the same generator as any value draws, so one seed reproduces a whole pipeline. An out-of-range R is an error, not a silently wrapped value.

3.4 Random values

`rand` and `randn` work for every format, and both accept an explicit RNG for reproducibility:

```
julia> rand(Xoshiro(1), Binary8p4se)
Binary8p4se(0.0703125 ≈ 0x21)

julia> rand(Xoshiro(8), Binary8p4se, 4)
4-element Vector{Binary8p4se}:
 Binary8p4se(0.40625 ≈ 0x35)
 Binary8p4se(0.021484375 ≈ 0x13)
 Binary8p4se(0.75 ≈ 0x3c)
 Binary8p4se(0.28125 ≈ 0x31)
```

Scalar `rand` takes a projection keyword. The default is a floor projection, which keeps results in $[0, 1)$ as `rand`'s contract requires; opting into `RTP_SN` or `RNE_SN` can return exactly 1.0, since mass near the top rounds up. The array and in-place forms always use the defaults — for an array under another projection, draw scalars explicitly.

3.5 Session defaults

Five process-wide defaults back the convenience forms: `DefaultType`, `DefaultReturnType`, `DefaultRoundingMode`, `DefaultSaturationMode`, and `DefaultProjection`. The projection default and its two components remain coherent regardless of which setter ran last:

```
DefaultProjection() == ProjSpec(DefaultRoundingMode(), DefaultSaturationMode())
```

A setter mutates state shared by every module in the process, and nothing scopes or restores it:

```
julia> w * two                                     # follows the default: RNE_SN → Inf
Binary8p4se(Inf ≈ 0x7f)

julia> DefaultProjection!(RTZ_SF);

julia> w * two                                     # every convenience call changed
Binary8p4se(224.0 ≈ 0x7e)

julia> DefaultProjection!(RNE_SN)                 # restore; the setting is process-wide
(NearestTiesToEven, SatNone)

julia> Multiply(Binary8p4se, RNE_SN, w, two)      # explicit p never reads a default
Binary8p4se(Inf ≈ 0x7f)
```

Library code and experiments should name `T` and `p` explicitly. The setters are appropriate for interactive sessions you control.

3.6 Interaction with Julia's numeric model

Same-format expressions use ordinary Julia spellings — `+` `-` `*` `/`, `exp`, `sqrt`, comparisons — each equivalent to one draft operation under the session default. Three boundaries are deliberate:

- **Mixed Binary formats do not promote.** No automatic rule can soundly pick a containing format (most format pairs have none within the grid, and pairwise selection is not associative), so cross-format arithmetic requires an explicit `Convert`.
- **Binary with an ordinary number promotes to the format's carrier.** `Binary8p4se(1.5) + 2.0` returns the `Float64` value 3.5, not a `Binary` value; the 72 wide formats promote to `BigFloat` instead, since a `Float64` cannot hold their datums. Project back explicitly when the result should be in the format.
- **Some inputs are refused rather than double-rounded.** `Convert` accepts every type it can project exactly: `Binary` values, machine floats, `Float128`, integers, `BigFloat`. A `Rational` would require a hidden second rounding to reach a carrier, so it throws with guidance — convert to a supported carrier first, making that rounding your explicit choice.

Two draft semantics differ from IEEE 754 by design. `Divide(x, 0)` is NaN for every `x`, including $\pm\text{Inf}$, and `Recip( $\pm\text{Inf}$ )` is 0 — there are no signed-infinity quotients. And the total order places the single NaN first, below $-\text{Inf}$, so sorting a `Binary` vector leads with NaN.

3.7 Next

This chapter computed one value at a time. The same contract must hold for large arrays, fused reductions, and storage at true bit width, without acceleration changing any code point. That is the subject of [Data at Scale](#).

Chapter 4

Data at Scale

One rule governs this chapter: acceleration may change how work is scheduled, never which code point is returned. Arrays, cached tables, fused block reductions, and packed storage are four schedules for the same exact-then-project computation.

4.1 Arrays and tables

Every operation has array methods with the scalar argument shape, operands replaced by arrays. Each element is exactly the scalar answer:

```
julia> A = Binary8p4se.([-1.0, 0.5, 2.0]);

julia> Exp(Binary8p4se, RNE_SN, A)
3-element Vector{Binary8p4se}:
 Binary8p4se(0.375 ≈ 0x34)
 Binary8p4se(1.625 ≈ 0x45)
 Binary8p4se(7.5 ≈ 0x57)
```

Under a deterministic projection, a unary or binary operation is a finite function on code points, so the package may cache it. The first array call for a signature can build a result table; later calls gather from it. Every table entry is produced by the scalar path and compared against it by the test gates, so declining the table — stochastic specs draw per element, and large signatures exceed the byte budget — means running the same scalar path per element, never computing a different answer. `table_policy` reports which shape a signature receives and why. When benchmarking, measure the first (table-building) call separately from warm calls.

Sorting uses integer order keys and, for large vectors, an $O(n)$ counting sort. Order follows the draft's total order, with the single NaN first.

4.2 Choosing a format

Format choice is a measurement, not a guess. At fixed bitwidth, precision trades against range, and the trade should be evaluated on data shaped like yours. For a unit-scale Gaussian sample of 50 000 values projected through each 8-bit signed extended format under `RNE_SF`:

```
Binary8p5se  rmse = 0.0133  maxfinite = 15.0
Binary8p4se  rmse = 0.0266  maxfinite = 224.0
Binary8p3se  rmse = 0.0528  maxfinite = 49152.0
Binary8p2se  rmse = 0.103   maxfinite = 2.147483648e9
```

RMSE roughly doubles for each significand bit given up, while the finite range grows enormously. For this sample no format overflows, so the extra range is unused and the highest precision wins. On wider-spread data the comparison changes, which is why three numbers should accompany any quantization choice: aggregate error (RMSE), worst single error, and the overflow or clamp rate. A format can score well on the first and fail on either of the others. When the dynamic range genuinely exceeds what the precision budget allows, do not buy range with a wider element format — get it from a block scale, below.

Unsigned formats suit quantities that are never negative (probabilities, memberships, normalized scores); finite formats suit domains where an infinity would be meaningless. Both spend otherwise-idle code points on resolution.

The decision, as a checklist:

1. Never negative? Use an unsigned format (u).
2. Is an infinity a meaningful outcome? If not, use a finite format (f).
3. Run the three-number report (RMSE, worst error, overflow rate) on data shaped like yours; take the highest precision whose overflow rate is acceptable.
4. Range still exceeds the precision budget? Use a block scale, below — not a wider element format.
5. Storage- or bandwidth-bound? Pack, at the end (packing changes layout, not the quantization).

4.3 Blocks

When data spans more range than a narrow element format holds, widening the element spends bits on exponent that were wanted for precision. A `Block{B,FS,FE}` shares one scale (format FS) across B elements (format FE); each lane represents `scale × element`. This is the draft's MX-style scheme. Construct a block directly, or quantize a tuple and let the draft's algorithm derive the scale from the largest finite magnitude:

```
julia> Block(Binary8pluf(4.0), Binary8p4se(1.5), Binary8p4se(-0.75),
             Binary8p4se(2.0), Binary8p4se(0.5))
Block{4, Binary8pluf, Binary8p4se}(Binary8pluf(4.0 ≈ 0x82), (...))

julia> ConvertToBlockMaxAbsFinite(Binary8pluf, Binary8p4se, RNE_SN, RNE_SN,
                                   (Binary8p4se(100.0), Binary8p4se(-12.0), Binary8p4se(0.5), Binary8p4se(3.0)))
Block{4, Binary8pluf, Binary8p4se}(Binary8pluf(64.0 ≈ 0x86), (...))
```

The arguments read in order: scale format, element format, a projection for the scale, a projection for the elements, then the values. Here the algorithm chose scale 64 from the largest element, 100, and projected each element against it. `ConvertFromBlock(T, p, b)` reads a block back out as plain values, decoding each `scale × element` exactly and projecting once. Scale formats are conventionally unsigned with precision 1 (Binary8pluf): every datum is a power of two, so dividing by the scale is exact.

Block reductions are where the exactness contract does real work. A reduction decodes every lane exactly, accumulates exactly — an integer span filter proves a Float128 accumulator suffices, or an arbitrary-precision accumulator sized from the formats takes over — and projects once:

```
julia> b = Block(Binary8p1uf(0.5), (Binary8p4se(80.0), Binary8p4se(-0.375),
                                   Binary8p4se(-5.5), Binary8p4se(2.25)));

julia> SmallFloats.blockdecode(b)           # the exact lane values
(40.0, -0.1875, -2.75, 1.125)

julia> BlockReduceAdd(Binary8p4se, RNE_SN, b) # exact sum 38.1875, projected once
Binary8p4se(40.0 ≈ 0x6a)

julia> lanes = SmallFloats.blockdecode(b);

julia> foldl((a, v) -> Add(Binary8p4se, RNE_SN, a, Convert(Binary8p4se, RNE_SN, v)),
            lanes; init = zero(Binary8p4se)) # rounding at every step
Binary8p4se(36.0 ≈ 0x69)
```

The exact sum is 38.1875 and the correct projection is 40. Lane-by-lane accumulation lands on 36 — a defined-result error that grows with B. BlockDotProduct gives the same guarantee for fused products: on random 32-lane blocks, the entire difference from the Float64 dot product is the initial quantization of the inputs; accumulation contributes nothing.

One pitfall dominates block practice. ConvertToBlockMaxAbsFinite quantizes a tuple, deriving the scale from the largest finite magnitude — but it takes elements that are already Binary. If raw data is staged through the narrow element format first, SatFinite clamps the large values before the block scale can absorb them, and blocking gains nothing. Measured on a 64×64 matrix whose rows span roughly 2^{16} in scale: direct Binary8p4se conversion has relative error 0.616, element-format staging 0.617, and staging through the wide-range Binary8p2se before block-quantizing 0.109 — the staging format's own precision. Stage wide, then block-quantize. At B = 32 with an 8-bit scale, storage costs 8.25 bits per value.

4.4 Packed storage

A Vector{Binary5p2se} spends a full byte on each 5-bit code point. PackedVector(v) stores elements at true bit width. It is a complete AbstractVector{F} — indexing, iteration, collect — and vmap computes on it directly, unpacking tiles internally:

```
julia> n = 1_000_000;

julia> model = [Binary5p2se(UInt8(rand(0:31))) for _ in 1:n];

julia> pv = PackedVector(model);

julia> (sizeof(model), sizeof(pv.data))
(1000000, 625000)
```

The rule is store packed, compute unpacked; there is deliberately no in-place packed arithmetic. Packing pays when memory or bandwidth is the constraint — for example, a large quantized model held resident. For compute-only work an ordinary Vector{F} avoids the unpack cost.

4.5 Float16 and BFloat16

Binary16p11se resembles Float16, and Binary16p8se resembles BFloat16, but P3109's exponent bias differs from IEEE's by one (16 versus 15; 128 versus 127). Every code point therefore denotes a value a factor of two away from its IEEE counterpart; largest finite values are 65472 versus 65504, there is one NaN rather than 2046 encodings, and no negative zero.

```
julia> using SmallFloats.Formats

julia> Convert(Binary16p11se, RNE_SF, Float16(1.5))    # a conversion: 1.5 ↦ 1.5
Binary16p11se(1.5 ≈ 0x4200)

julia> reinterpret(Binary16p11se, Float16(1.5))        # the same bits: 1.5 ↦ 0.75
Binary16p11se(0.75 ≈ 0x3e00)
```

The reinterpreted value is wrong by a factor of two, with no error and no warning — the most dangerous operation in cross-format work, because the numbers look plausible. Use `Convert` across the IEEE boundary, never `reinterpret`. `datumsexact` reports which format pairs round-trip losslessly.

4.6 Next

A [Worked Example](#) composes this chapter and the two before it into one measured quantization task, end to end.

Chapter 5

A Worked Example

The preceding chapters in one task: choose a format for real data, quantize, measure what the choice cost, and record what produced the numbers. The pattern transfers directly to model weights, activations, or any signal.

5.1 The data, and an explicit policy

A fixed seed makes the session reproducible; RNE_SF — nearest, ties to even, clamp to finite — is named once and used throughout, so the report stays meaningful even if a session default changes elsewhere:

```
using SmallFloats, Random, Statistics

rng = Xoshiro(42)
w = randn(rng, 50_000) .* 0.25      # a typical trained-weight scale
```

5.2 Measure, with three numbers

One number is not enough evidence for a quantization choice. Aggregate error can look fine while single values are badly wrong, and both can look fine while the tails overflow. Report all three:

```
function quantization_report(::Type{F}, x; projection = RNE_SF) where {F}
    q = [Convert(F, projection, xi) for xi in x]
    y = decode.(q)
    finite = isfinite.(y)
    (
        format = formatname(F),
        rmse = sqrt(mean((x[finite] .- y[finite]).^2)),
        max_error = maximum(abs.(x[finite] .- y[finite])),
        nonfinite_rate = count(!isfinite, y) / length(y),
        boundary_rate = count(v -> abs(v) == decode(MaxFiniteOf(F)), y) / length(y),
    )
end

quantization_report(Binary8p4se, w)
```

```
(format = :Binary8p4se, rmse = 0.0066, max_error = 0.0622,
 nonfinite_rate = 0.0, boundary_rate = 0.0)
```

Average damage is small, the worst single value is off by ten times the average, and nothing overflowed — for this sample, range was never the binding constraint.

5.3 Compare candidates under one policy

Same sample, same projection, four formats — never change the data and the policy in the same experiment:

```
for F in (Binary8p5se, Binary8p4se, Binary8p3se, Binary8p2se)
    r = quantization_report(F, w)
    println(rpad(String(r.format), 12), " rmse=", round(r.rmse; sigdigits=3),
        " max=", round(r.max_error; sigdigits=3),
        " nonfinite=", r.nonfinite_rate, " boundary=", r.boundary_rate)
end
```

Binary8p5se	rmse=0.00356	max=0.0256	nonfinite=0.0	boundary=0.0
Binary8p4se	rmse=0.00662	max=0.0622	nonfinite=0.0	boundary=0.0
Binary8p3se	rmse=0.0132	max=0.0881	nonfinite=0.0	boundary=0.0
Binary8p2se	rmse=0.0261	max=0.162	nonfinite=0.0	boundary=0.0

RMSE roughly doubles per significant bit given up, exactly as the datum spacing does. Every format holds this data without overflow, so the range the lower-precision formats offer goes unused and Binary8p5se wins outright. On wider-spread data the last two columns stop reading zero and the ranking changes — which is what they are there to catch.

5.4 Inspect the worst misses

```
q = [Convert(Binary8p4se, RNE_SF, x) for x in w]
err = abs.(w .- decode.(q))
worst = partialsortperm(err, 1:3; rev = true)
[(w[i], decode(q[i]), err[i]) for i in worst]
```

```
(-1.06279, -1.125, 0.06221)
(-1.06092, -1.0, 0.06092)
(-1.06967, -1.125, 0.05533)
```

The worst errors cluster just past 1.0, where Binary8p4se's spacing widens to 0.125 at the binade boundary — a structural property of the format, not an unlucky sample. Knowing where the errors live says whether a different format would help or the data needs rescaling. If failures cluster by row or channel scale instead, use a block scale (see [Data at Scale](#)).

5.5 Record what produced the result

```
d = conformance_dict()           # formats, operations, specializations, approximations
rev = draft_revision()          # "IEEE P3109/D1, uploaded 2026-07-17"
```

Serialize `d` with any JSON or TOML writer and store it, the draft revision, the chosen format, the projection name, and the seed beside the quantized artifact. A later reader can then reproduce the numbers or explain a difference without guessing what the session looked like.

5.6 Next

[Assurance](#) covers the record-keeping used above in full: the provenance test, measured approximation, and how the defined path itself is verified.

Chapter 6

Assurance

When two runs disagree, exactly three causes are legitimate: the policy changed, the policy itself requests randomness, or a registered approximation was selected. Anything else is a defect. This chapter provides the tools that make that determination and the records that support it: the provenance test, the measured-approximation registry, the exportable conformance declaration, and a summary of how the defined path is verified.

6.1 The provenance test

When two runs differ, check in order:

1. Did the operation, result format, or projection specification change?
2. Is the projection stochastic, and did the draw or RNG stream change?
3. Did the call explicitly select a registered approximation?
4. Did process-global defaults change between calls?

If all four answers are no, a changed code point is a defect and should be reported as one. For experiments that must be reproducible, record the result format, the projection specification, the RNG algorithm and seed, and any approximation's name and measured bound alongside the results.

6.2 Measured approximation

The default API never substitutes an approximation. A faster, inexact kernel enters only through the registry, and the registry measures its worst-case deviation — in code points along the total order, exhaustively where the input space permits — rather than accepting a declared figure:

```
julia> one4 = Binary8p4se(1.0);

julia> hardtanh(x) = Clamp(Binary8p4se, RNE_SN, x, Negate(one4), one4);

julia> κ, exhaustive = measure_kappa(hardtanh, :Tanh, Binary8p4se,
                                   (Binary8p4se,), RNE_SN)
(4.0, true)           # worst deviation: 4 code points, all 256 inputs checked
```

A declaration that understates the measurement is rejected, and no partial registration state remains:

```
julia> register_approx!(:hardtanh_act, :Tanh, Binary8p4se,
                       (Binary8p4se,), RNE_SN, hardtanh; κ=1)
ERROR: ArgumentError: declared κ = 1.0 understates measured κ = 4.0 – registration rejected
```

Registered with a truthful bound, the implementation is retrieved by name (`approx`, `list_approx`), its `k` is queryable, and it appears in the conformance declaration automatically. Code-point bounds also support decision rules: two defined results more than $2k$ apart cannot have their comparison inverted by the approximate evaluator, which makes pruning with a cheap evaluator provably sound.

6.3 The conformance declaration

`conformance()` returns a live statement of the session: formats, operations, mode vocabulary, the table specializations built so far, and every registered approximation. `conformance_dict()` is the plain-Dict form for JSON or TOML export:

```
d = conformance_dict()
(d["package"], length(d["formats"]), length(d["operations"]),
 [s["op"] for s in d["cached_specializations"]])
```

```
("SmallFloats.jl 0.4.0", 504, 52, Any[])
```

The first three entries are package properties: 504 formats, 52 operations, always. The fourth is session-dependent — empty here because this session had made no array call yet — so snapshot the declaration after the work, not at the top of a script. `draft_revision()` reports the implemented draft revision (IEEE P3109/D1, uploaded 2026-07-17); include it with any exported artifact.

6.4 Verifying your own code

The same exhaustive method is available to application code, because format input spaces are small enough to enumerate. A stochastic-rounding distribution, for example, is checkable exactly rather than statistically: the value $2 + 3/64$ sits at fraction $3/16$ of an ulp, so `StochasticA{4}` must round it up for exactly 3 of the 16 draws —

```
σ4 = RSA_SN(4)
x = 2.0 + 3/64
count(decode(SmallFloats.project(Binary8p4se, σ4, x; R)) == 2.25 for R in 0:15)
```

```
3
```

Sweeping `R` turns a statistical question about bias into an exhaustive one.

Ordering guarantees are checkable the same way. Ranking and decision systems consume scores through order, so a conversion between formats must be monotone — and with a few hundred datums per format, that is provable by walking every value in total order and confirming its image never goes backward:

```
using SmallFloats: order_key

function monotone_conversion(::Type{From}, ::Type{To}) where {From,To}
    prev = nothing
    for v in sort(From.(0x00:UInt8(2^bitwidth(From) - 1)))
        isnan(decode(v)) && continue
        g = Convert(To, RNE_SN, v)
        prev !== nothing && order_key(g) < order_key(prev) && return false
        prev = g
    end
    true
end
monotone_conversion(Binary8p3se, Binary8p4se)
```

```
true
```

A ranking pipeline whose conversions all pass this audit cannot invert a preference by changing formats. The same pattern — enumerate the inputs, compare against an independently computed reference — verifies quantizers and custom fused kernels; the full manual's [Verify Custom Code](#) page gives worked recipes.

6.5 How the defined path is verified

The test suite enumerates rather than samples wherever the input space allows, and reports "sampled" explicitly where it does not. Because formats are finite, most correctness claims are checked by exhaustion: every code point of every format round-trips; cached tables compare entry-for-entry with the scalar path; the optimized rounding core compares against the generic one; and two independent high-precision references (Float128-first and pure-MPFR builds) must agree bit-for-bit. Disabling the Float128 machinery (`ENV["SmallFloats_Float128"] = "disable"`) is itself a tested equivalence: identical results, different speed.

Structurally, there is one write path. Every code point the package produces passes through `RoundToPrecision` → `Saturate` → `Encode`, fed by exact arithmetic on a carrier the format selects — `Float64` where exact, `Float128` above it, an exact dyadic carrier for the widest formats, with MPFR enclosures deciding the remainder. No second rounding routine exists, and nothing approximate is reachable outside the registry. The full manual's [Technical Guide](#) covers the architecture, the oracle's rigor classes, and the verification gates in detail.

6.6 Next

[Quick Reference](#) collects the names and signatures from all four chapters in brief.

Chapter 7

Quick Reference

Names and signatures in brief. For semantics, see the preceding chapters; for exhaustive tables, the full manual's Reference section.

7.1 Values in and out

```
Binary8p4se(1.6)      # Real → project to nearest datum
Binary8p4se(0x45)     # Unsigned → CODE POINT (validated)
Convert(T, p, x)       # explicit projection, numeric for every integer
decode(x)              # the exact datum, on the format's carrier
codepoint(x)           # the stored identity
T(codepoint(x)) == x   # the round-trip law
```

7.2 Format queries

Pure functions of the type parameters; each takes a type or a value, and folds to a constant either way.

```
bitwidth(T)  precision(T)  issigned(T)  isextended(T)
expbias(T)   expbitwidth(T) trailingsigbits(T)
MaxFiniteOf(T)  MinFiniteOf(T)  MinPositiveOf(T)
MinNormalOf(T)  MaxSubnormalOf(T)
format(K, P, SGN, EXT)    # programmatic route to any of the 504
```

Draft-named twins exist throughout: BitwidthOf, PrecisionOf, ExponentBiasOf,

7.3 Projection specifications

`ProjSpec(rounding, saturation)`; every deterministic pairing is predefined as `R<mode>_S<mode>`:

	SatNone	SatFinite	SatPropagate
NearestTiesToEven	RNE_SN	RNE_SF	RNE_SP
NearestTiesToAway	RNA_SN	RNA_SF	RNA_SP
TowardPositive	RTP_SN	RTP_SF	RTP_SP
TowardNegative	RTN_SN	RTN_SF	RTN_SP
TowardZero	RTZ_SN	RTZ_SF	RTZ_SP
ToOdd	RT0_SN	RT0_SF	RT0_SP

Stochastic constructors: `RSA_SN(N)`, `RSB_SF(N)`, `RSC_SP(N)`, ... — families A/B/C crossed with the saturation modes, N random bits (default 8). Pass `rng` for a stream or $R \in 0:2^N-1$ for one exact draw. `RNE_SN` is the initial session default.

7.4 Operations

52 draft operations — 30 unary, 18 binary, 3 ternary, plus `Convert` — every one spelled `Op(fr, p, operands...)` with a same-format convenience `Op(x...)`, array methods, and `Block/Scaled` variants. Families: sign and arithmetic (`Abs ... Divide`, fused `FMA/FAA`), roots and `exp/log`, trig, hyperbolic, π -scaled trig, `Softplus/Hypot/ArcTan2`, extrema and `Clamp`, comparisons and `TotalOrder`, format queries, `NextGreaterThan / NextLessThan`. Base spellings (`+`, `exp`, `sqrt`, `min`, `nextfloat`, ...) map one-to-one onto same-format calls under the session default.

7.5 Session defaults

Read with `DefaultX()`, set with `DefaultX!(v)`:

<code>DefaultType</code>	<code>DefaultReturnType</code>	
<code>DefaultRoundingMode</code>	<code>DefaultSaturationMode</code>	<code>DefaultProjection</code>

The projection default and its two component defaults stay coherent no matter which setter ran last. All five are process-global; name `T` and `p` explicitly in anything that must be reproducible.

7.6 Display

`set_show_style!(s) / get_show_style()`, or per-stream `I0Context(io, :binary_show_style => s)`, with $s \in (:value, :codepoint, :datum, :typed)$ — e.g. `1.625 · 0x45 · 1.625 ≈ 0x45 · Binary8p4se(1.625 ≈ 0x45)`. Presentation only; no result ever depends on it.

7.7 Blocks, storage, conformance

```
Block(scale, elems...)           # scale × element per lane
ConvertToBlockMaxAbsFinite(FS, FE, ps, pe, elems)
BlockReduceAdd(T, p, b); BlockDotProduct(T, p, a, b)
PackedVector(v)                 # true bit-width storage
measure_kappa(f, op, T, argTs, p) # exhaustive κ measurement
register_approx!(name, op, T, argTs, p, f; κ)
conformance_dict(); draft_revision()
```

7.8 Common mistakes

1. `T(2)` where `T(0x02)` was meant, or the reverse. The integer type's signedness selects value versus code point, and both calls succeed.
2. Vector of the abstract `Binary{K,P,SGN,EXT}`. The abstract format boxes every element; use the concrete alias.
3. A format or spec held in a non-const global. Every call then pays dynamic dispatch (about 1 μ s), and benchmarks measure the dispatcher rather than the package.
4. Reading `SatNone` as a clamp. Only `SatFinite` guarantees a finite result; `SatNone` applies the draft's rows and can produce $\pm\text{Inf}$ or `NaN`.
5. Staging raw data through the narrow element format before block-quantizing. The clamp then happens before the block scale can absorb it; stage through a wide-range format instead.

Chapter 8

Troubleshooting

Symptom-first. Each entry names what you see, why, and the fix.

T(0x02) and T(2) give different values, and it looks like a bug. An Unsigned argument is a code point; every other integer is a value to project. Both calls are valid, so nothing warns. Fix: `T(Int(c))` when you mean the number, `T(c::Unsigned)` when you mean the code point, or `Convert(T, p, x)` when intent must be explicit. See [Formats and Values](#).

An array of Binary values is slow, or every element is boxed. The element type is the abstract parametric format, which is not `isbits`. Fix: use a concrete alias as the element type (`Vector{Binary8p4se}`), or obtain one from `format(...)` when the parameters are runtime values.

A benchmark reports about 1 μ s per call for a warm scalar operation. The format or spec was read from a non-const global, so every call pays dynamic dispatch; the benchmark measures the dispatcher. Fix: pass formats as type parameters or const bindings, and verify zero warm-path allocation before trusting a number.

Overflow produced Inf where a clamp was expected (or the reverse). `SatNone` is not a clamp; it applies the draft's domain-, sign-, and direction-dependent rows, which can yield $\pm\text{Inf}$, an extremal finite value, or NaN. Only `SatFinite` guarantees a finite result. See [Computing](#).

Mixing two Binary formats throws a promotion error. Deliberate: no automatic rule can soundly pick a containing format. Fix: `Convert(T, p, x)` to the format you want before combining.

Constructing from a Rational throws. Also deliberate: it would require a hidden second rounding. Fix: convert to a supported carrier first (`T(Float64(r))`) and own that rounding.

A stochastic pipeline gives different results each run. Stochastic projections draw random bits. Fix: pass a seeded rng for a reproducible stream, or an explicit `R` in `0:2^N-1` for bit-exact single projections in tests. Thread one rng through every call that must agree; constructing a second generator mid-pipeline desynchronizes the streams.

The first array call is much slower than every later one. Deterministic unary/binary calls may build a cached result table on first use; you measured the build. Fix: warm the exact specialization first; benchmark cold and warm separately. `table_bytes()` inspects the cache, `empty_tables!()` resets it.

Float16/BFloat16 data comes through at half or double its value. Bits were reinterpreted across an exponent-bias difference of one. Fix: always `Convert`, never `reinterpret`, across the IEEE boundary. See [Data at Scale](#).

Block quantization did not help data with wide per-row scales. The raw values were staged through the narrow element format first, so `SatFinite` clamped them before the block scale could absorb the range. Fix: stage through a wide-range format, then `ConvertToBlockMaxAbsFinite`.

sort puts NaN first, unlike Float64 sorting. Not a bug: the draft's total order places the single NaN below $-\text{Inf}$. Sort with `rev=true` if you need it last.

Division by zero returned NaN, not Inf. P3109 defines `Divide(x, 0)` as NaN for every `x`, and `Recip( $\pm\text{Inf}$ )` as 0. Check divisors explicitly if an algorithm relies on IEEE signed-infinity quotients.

Chapter 9

Glossary

One line per term, with the chapter that develops it.

Term	Meaning	Chapter
datum	One exact value in a format's finite set.	Values
code point	The integer naming a datum; what a value stores.	Values
code unit	The storage word for code points: UInt8 at $K \leq 8$, UInt16 above.	Values
format	A finite datum set with an encoding: Binary{K,P,SGN,EXT}.	Values
alias	The concrete named subtype (Binary8p4se) code should use.	Values
binade	The span between consecutive powers of two; datum spacing doubles per binade.	Values
ulp	The datum spacing at a given magnitude (unit in the last place).	Values
projection	The single rounding-plus-saturation step from exact result to code point.	Computing
ProjSpec	A named (rounding mode, saturation mode) pair, e.g. RNE_SN.	Computing
SatNone	The draft's full boundary rows — not "no saturation".	Computing
stochastic mode	Rounding whose landing depends on a random draw R; a defined distribution.	Computing
session default	One of five process-global settings behind + and friends.	Computing
datum carrier	The exact type decode returns: Float64, Float128, or dyadic.	Values
promotion carrier	Where Binary + ordinary number lands: Float64, or BigFloat for wide formats.	Computing
defined result	The draft-required code point for an operation, format, and spec.	Assurance
table	A cached finite operation, entries produced by the scalar path.	Data at Scale
block / scale	B elements sharing one scale; each lane is $\text{scale} \times \text{element}$.	Data at Scale
span filter	The integer test proving a Float128 accumulation is exact.	Data at Scale
PackedVector	Storage at true bit width; compute unpacks internally.	Data at Scale
κ (kappa)	A registered approximation's measured worst-case error, in code points.	Assurance
total order	The draft's complete ordering; the single NaN sorts first.	Values
exhaustive	Checked over every member of the input space, as opposed to sampled.	Assurance